

SupplyGuard: A Temporal Graph Neural Network Framework for Predictive Supply Chain Disruption Risk Propagation

Vemula Sanath Chandra

Dept. of Computer Science and Engineering
Anurag University
Hyderabad, India
sanathchandra2005@gmail.com

Malyala Sai Bharadwaj

Dept. of Computer Science and Engineering
Anurag University
Hyderabad, India
saimalyala7@gmail.com

Swami Sriharsha

Dept. of Computer Science and Engineering
Anurag University
Hyderabad, India
sriharshaswamy@gmail.com

Under the guidance of Mrs. N. Chandana, Assistant Professor, Dept. of Computer Science and Engineering, Anurag University

Abstract—In a supply network with several echelons, trouble that starts upstream rarely stays there; it tends to move down to later tiers. Most risk forecasters, however, output one aggregated index and are tested at horizons so short that simply repeating the last observation is hard to outperform. This paper introduces SupplyGuard, a spatio-temporal network that produces a separate risk-index forecast for the Supplier, Manufacturer, Distributor and Retailer echelons, five steps ($H = 5$, nominally ten minutes) into the future. The architecture chains two relational graph-convolution layers, each with distinct weights for upstream and downstream neighbours, into a two-layer LSTM reused by every node, and ends in a residual head that outputs only the change relative to the latest reading; Integrated Gradients is applied after training to attribute each node's forecast. On the Mendeley SCRM data (592,599 rows kept after cleaning, a time-ordered 80/10/10 split and 57,875 test windows), the directed variant attains a mean per-node MSE of $2.73 \times 10^{-4} \pm 0.03 \times 10^{-4}$ across five seeds. That is 31% lower than Naive Persistence (3.97×10^{-4}) and 4% lower than an LSTM without the graph (2.84×10^{-4}). Most of this improvement comes from shock windows, on which the model's error falls to roughly 50% of the persistence error; on calm windows the ratio reverses and its error is roughly double. Persistence keeps the edge in three-tier severity labelling (83.6% versus 81.3%). Attributions from Integrated Gradients reach a median completeness gap of 9.4×10^{-5} ; they quantify the local sensitivity of the trained model and carry no causal meaning. Limitations include a confounded graph-free ablation and a single linear topology.

Keywords—Supply chain networks, graph convolutional networks, spatio-temporal modeling, ripple effect propagation, LSTM recurrence, explainable artificial intelligence, path-integrated gradients, residual risk forecasting.

I. INTRODUCTION

Present-day production and trade depend on lean stock policies, just-in-time (JIT) delivery and closely coordinated transport. These practices reduce holding costs in steady state but leave little buffer against disturbances. A production outage, a missing component or a congested port at an upstream tier shifts schedules, and the shift can travel through intermediate plants and distribution hubs until shelves run empty at the retail end. Supply chain researchers call this spreading the ripple effect [1], [7].

In response, operations teams increasingly use continuous telemetry of material transit, machine status, inventory levels and logistics cost. Forecasts are usually produced from these streams by autoregressive estimators, discrete-event simulation or Long Short-Term Memory (LSTM) networks [4], and machine learning is spreading across supply chain management in general [12]. We address two weaknesses of these approaches.

First, several of them, among them the hybrid graph-LSTM of [10] that we rebuild as a baseline, reduce the whole network to one risk number. Such a number hides whether a bottleneck

began at a supplier or at a distributor, which leaves planners unable to aim their response. Second, short-horizon forecasts are easy to match with a trivial baseline: in the dataset used here, adjacent two-minute observations are very strongly correlated (lag-1 Pearson r between 0.97 and 0.996 across the four echelons), so predicting that the future equals the present is difficult to beat. A forecasting model must therefore be judged against persistence.

A. Motivation

- **Thin Buffers in Lean Chains:** Because lean chains keep little reserve stock, one supplier's bottleneck can carry on to the echelons after it [1].
- **Limited Visibility in Aggregate Indices:** A single network-wide index does not show which echelon is deteriorating.
- **Why Ten Minutes Ahead:** A five-step target ($H = 5$, 10 minutes) gives planners more warning than a one-step target ($H = 1$, 2 minutes) while keeping the comparison with persistence direct.
- **The Need for Interpretable Output:** Operators must see which inputs drive a forecast. Integrated Gradients supplies signed sensitivities for each node whose completeness can be measured; they remain sensitivities rather than causal explanations.

B. Objectives

01. Forecast the risk indices of the four echelons concurrently, ten minutes ($H = 5$ steps) ahead, from a ten-step window.
02. Represent directed material flow with a graph layer that treats upstream and downstream messages separately.
03. Benchmark the model against Naive Persistence, Ridge-AR(10) and a graph-free LSTM under one chronological split and five seeds.
04. Attach Integrated Gradients attributions to each forecast and report their completeness gap and a deletion test.
05. State where the learned model adds value over persistence and where it does not.

C. Key Methodological Contributions

- **Directed Spatio-Temporal Architecture:** Every time step passes through a relational graph convolution holding distinct self, upstream and downstream weight matrices, after which one LSTM, reused across nodes, reads the sequence (Section III-D).
- **Concurrent Node-Level Forecasts:** A single forward pass gives the four echelon forecasts, from which the mean risk index is derived.

- **Residual Persistence Anchoring:** The head predicts the change from the current state, so the model starts from the persistence forecast [18].
- **Evaluation Against Persistence:** Five-seed results with standard deviations against Naive Persistence, Ridge-AR(10) and a graph-free LSTM, reported separately for shock and calm windows, plus a measured completeness check for Integrated Gradients [8].

D. Problem Statement and Formal Framing

We represent the multi-echelon supply network as a directed graph $G = (V, E)$, where $|V| = 4$ nodes are the Tier-1 Supplier (S), Manufacturer (M), Regional Distributor (D) and Final Retailer (R), and $E = \{(S, M), (M, D), (D, R)\}$ is the direction of material flow. At each time step t the system observes x_t in $\mathbb{R}^5$: the min–max scaled risk indices of the four echelons and the concurrent Total Cost. Given a lookback window of $L = 10$ consecutive observations, the input is X in $\mathbb{R}^{(L \times 5)}$. The task is to learn a mapping f_θ from X to the risk vector $y_{\text{hat}}(t+H)$ in $\mathbb{R}^4$ at horizon $H = 5$ steps (nominally 10 minutes), together with, for any selected target node k , an attribution matrix Attr in $\mathbb{R}^{(L \times 5)}$ giving the local sensitivity of node k 's forecast to every past input.

II. LITERATURE SURVEY

A. Disruption Cascades and Ripple Effect Modeling

Ripple-effect research has been studied through analytical and simulation lenses. Ivanov, Dolgui and Sokolov [1] analysed how digital technology and Industry 4.0 affect ripple-effect and supply chain risk analytics. Dolgui, Ivanov and Sokolov [7] reviewed quantitative ripple-effect research, separating it from the bullwhip effect and grouping the approaches into optimisation, simulation, control-theoretic and reliability methods. Simchi-Levi et al. [2] describe Time-to-Survive (TTS) and Time-to-Recover (TTR) as measures of how long a network can keep operating, and how long recovery takes, when a node fails.

B. Sequence Learning and Graph Neural Architectures

To handle high-frequency telemetry, researchers adopted recurrent networks. Hochreiter and Schmidhuber [4] introduced the Long Short-Term Memory (LSTM) network, which mitigates vanishing gradients over long sequences. A plain LSTM over the stacked input does not encode which facility supplies which. Scarselli et al. [16] introduced the graph neural network model, and Kipf and Welling [6] simplified it to the Graph Convolutional Network (GCN) for semi-supervised node classification. Yu, Yin and Zhu [5] combined spatial graph convolutions with temporal convolutions for traffic forecasting (STGCN), and Li et al. [17] modelled traffic as diffusion on a directed graph inside a recurrent network. Wu et al. [9] survey graph neural networks, and Velickovic et al. [13] introduced graph attention networks.

Farzhana and Dev Harris [10] fuse a graph convolution with an LSTM to predict supply chain risk on the same dataset used here. We re-implement that fusion as a baseline (global mean

pooling of the graph output to a scalar risk index) and extend it to node-level forecasting; the pooling and layer sizes of our re-implementation are our interpretation of the published description, not a verbatim specification.

C. Explainability

Attribution methods require care. Adebayo et al. [15] showed that several saliency methods can produce maps that are insensitive to the model's parameters or to the data labels, which motivates sanity checks for any attribution. Sundararajan, Taly and Yan [8] proposed Integrated Gradients (IG), which accumulates gradients along a straight path from a baseline and satisfies Sensitivity and Implementation Invariance; completeness (the attributions sum to the difference in model output) follows. Lundberg and Lee [19] unify additive feature-attribution methods, an alternative that we do not use. We apply IG to the directed graph model and report its completeness gap and a deletion test.

TABLE I
SUMMARY OF PRIOR APPROACHES AND METHODOLOGICAL LIMITS

Ref.	Year	Methodology	Evaluation Scope	Key Practical Limitation
[7]	2018	Literature review (optimisation, simulation, control, reliability)	Ripple-effect research	Review; no learned telemetry forecaster
[2]	2014	Time-to-Survive / Time-to-Recover analysis	Supplier mapping	Analytical; not designed for minute-level telemetry
[10]	2025	Hybrid GNN-LSTM	Mendeley SCRM telemetry	Re-implemented here with a scalar risk output
[5]	2018	Spatio-temporal graph convolution	Road sensor networks	Traffic forecasting; not evaluated on supply chains
[8]	2017	Integrated Gradients	Image, text and other models	Needs a scalar target and a chosen baseline
[17]	2018	Diffusion convolutional recurrent network	Traffic forecasting	Directed diffusion on road graphs; not applied to supply chains

D. Research Gap

The work reviewed above leaves three gaps that this paper addresses. First, the hybrid GNN-LSTM for this dataset [10] outputs one aggregated risk index, so the affected echelon is not identified. Second, spatio-temporal graph models such as [5] are built for symmetric road networks, while material flow in a supply chain is directed; we therefore compare a symmetric (Kipf–Welling [6]) and a directed variant. Third, forecasts should be reported against persistence, which is strong at short horizons. We do not claim that attribution identifies a root cause.

III. PROPOSED METHODOLOGY / IMPLEMENTATION

A. System Architecture & Pipeline Overview

SupplyGuard has five stages, shown in Fig. 1 and Fig. 2: cleaning and windowing of the telemetry, two graph-convolution layers applied at every time step, a shared LSTM

per node, a residual head that forecasts the four risk indices five steps (nominally ten minutes) ahead, and post hoc Integrated Gradients attribution.

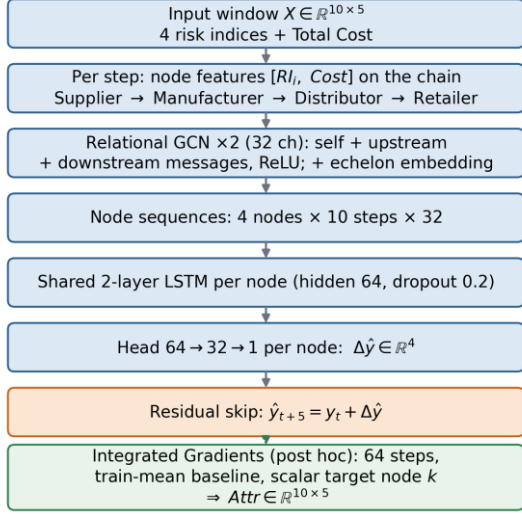


Fig. 1. SupplyGuard model: two relational graph-convolution layers with self, upstream and downstream messages, a shared per-node LSTM, a residual head, and the Integrated Gradients module.

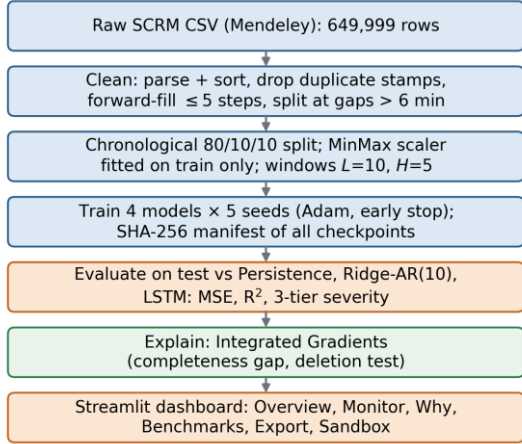


Fig. 2. Processing pipeline: cleaning, chronological split with train-only scaling, five-seed training, evaluation against baselines, Integrated Gradients, and the Streamlit dashboard.

B. Software Engineering & Component Architecture

Figure 3 shows the module structure of the implementation. `src/dataset.py` builds leakage-safe windows (chronological split; scaler fitted on the training partition only), `src/graph_builder.py` builds the symmetric and directed adjacency matrices, and `src/models/graph_layers.py` implements the graph convolution natively in PyTorch (no PyTorch Geometric). `src/models/st_gcn_lstm.py` defines the proposed model and the neural baselines, `training/` trains and evaluates them, `src/explainability.py` implements Integrated Gradients, and a Streamlit application reads the saved outputs.

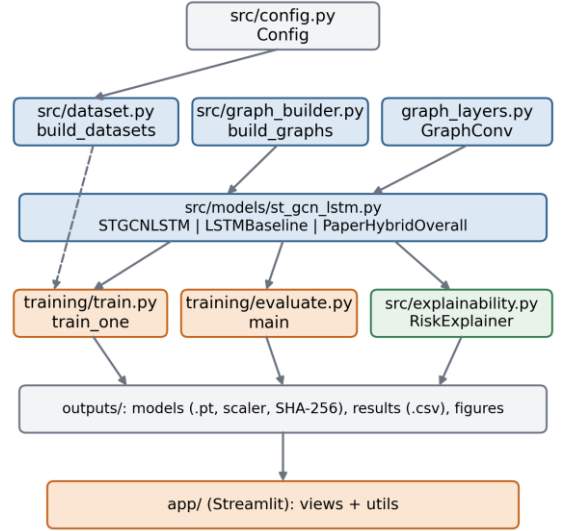


Fig. 3. Module dependencies of the implementation (`src/`, `training/`, `app/`).

C. Dataset Provenance and Preprocessing Protocol

We use the Mendeley time-series dataset for risk assessment in supply chain networks [3] (file `SCRM_timeSeries_2018_train.csv`). It has 649,999 rows between 28 Jan 2015 and 19 Dec 2018, with records for 2015, 2016 and 2018 only. Each row holds a timestamp, the risk index of each of the four echelons and Total Cost, nominally every two minutes. Cleaning removes 2,363 duplicate timestamps, forward-fills gaps of at most five steps, drops 53,960 rows that remain incomplete and 1,077 rows in segments shorter than 15 steps, and splits sequences wherever the time gap exceeds six minutes. This leaves 592,599 rows (91.2%). The cleaned rows are split chronologically 80/10/10 (474,079 / 59,259 / 59,261 rows). A window of $L = 10$ rows predicts the risk indices five rows later, and windows never cross a segment boundary, so every target lies inside its own partition; this gives 460,135 training, 57,817 validation and 57,875 test windows, with the test windows spanning 17 Sep to 19 Dec 2018. Because windows are indexed by rows, the horizon is five rows; its median duration is 10.0 minutes and its mean 10.8 minutes. The min-max scaler is fitted on the training partition only. Before cleaning, 5.1% of Distributor values and 5.5% of Total Cost values are missing.

TABLE II
MENDELEY SCRM TELEMETRY CHANNELS AND TRAINING-PARTITION STATISTICS (RAW UNITS)

Supply Echelon	Node ID	Monitored Telemetry Channel	Mean (μ)	Std Dev (σ)
Tier-1 Supplier	Node S	Risk index RI_Supplier1	1.701	0.041
Manufacturer	Node M	Risk index RI_Manufacturer1	2.711	1.134
Regional Distributor	Node D	Risk index RI_Distributor1	2.452	0.705
Final Retailer	Node R	Risk index RI_Retailer1	2.413	0.252
Logistics Network	Edge Set	Total_Cost	87.606	76.607

D. Mathematical Model Formulation

1) Relational graph convolution: Let A in $\{0,1\}^{(4 \times 4)}$ with $A_{(i,j)} = 1$ if facility i ships to facility j (edges $S \rightarrow M$, $M \rightarrow D$, $D \rightarrow R$). We define $A_{\text{down}} = \text{rownorm}(A^T)$, which averages messages from upstream parents, and $A_{\text{up}} = \text{rownorm}(A)$,

which averages messages from downstream children. At each time step the node features $H^{(0)}$ in $R^{(4 \times 2)}$ hold each node's risk index and the Total Cost. Each layer computes

$$H^{(l+1)} = \text{ReLU}(H^{(l)} W_{\text{self}}^{(l)} + A_{\text{down}} H^{(l)} W_{\text{down}}^{(l)} + A_{\text{up}} H^{(l)} W_{\text{up}}^{(l)} + b^{(l)}) \quad (1)$$

where W_{self} , W_{down} , W_{up} in $R^{(F_{\text{in}} \times F_{\text{out}})}$ are learnable, b is a bias and $F_{\text{out}} = 32$. Two layers are stacked so that information can travel two hops (S can influence D). A learnable echelon embedding is added to the output, $H^{(2)}$ in $R^{(4 \times 32)}$. The symmetric variant uses $\text{ReLU}(A_{\text{hat}} H W + b)$ with $A_{\text{hat}} = D^{(-1/2)}(A_{\text{sym}} + I)D^{(-1/2)}$ [6].

2) Recurrent temporal memory: For each node the $L = 10$ embeddings form a sequence in $R^{(10 \times 32)}$. A two-layer LSTM (hidden size 64, dropout 0.2) with weights shared across the four nodes maps each node's sequence to its final hidden state:

$$h_{t'}^{(i)} = \text{LSTM}(H^{(2)}_{(\text{tau}, i)} : \text{tau} = t-L+1, \dots, t) \quad (2)$$

$h_{t'}^{(i)}$ in R^{64} . Total Cost enters through the node features; it is not concatenated again after the graph layers.

3) Residual persistence head: A perceptron ($64 \rightarrow 32 \rightarrow 1$, ReLU, dropout 0.2) maps $h_{t'}^{(i)}$ to a correction, and the forecast adds it to the current value [18]:

$$y_{\text{hat}}_{(t+H)}^{(i)} = x_{t'}^{(i)} + g(h_{t'}^{(i)}) \quad (3)$$

With a zero correction the model reproduces persistence, so any gain must come from the learned correction.

4) Training objective: The four echelons are predicted jointly, a form of multi-task learning [14], by minimising the mean squared error. No explicit weight decay is used; early stopping on validation loss is the regulariser.

$$L(\theta) = (1/|V|) \sum_{i=1..|V|} (y_{\text{hat}}_{(t+H)}^{(i)} - y_{(t+H)}^{(i)})^2 \quad (4)$$

5) Integrated Gradients: For a window X and a target node k , the scalar output is $f_k(X) = y_{\text{hat}}_{(t+H)}^{(k)}$. With baseline $X^{(0)}$ (the training-mean feature vector repeated over the window) and $M = 64$ steps, the attribution is given below. Completeness requires $\text{SUM Attr} = f_k(X) - f_k(X^{(0)})$; we report the residual. Attributions describe local sensitivity along this path and are not causal effects.

$$\text{Attr}(X) = (X - X^{(0)}) \odot (1/M) \sum_{m=1..M} \partial f_k / \partial X |_{(X^{(0)} + (m/M)(X - X^{(0)}))} \quad (5)$$

Algorithm 1: SupplyGuard Forecasting & Attribution

Input: window X in $R^{(L \times 5)}$ (columns 1–4 risk indices, column 5 Total Cost), $L = 10$, $H = 5$; A_{down} , A_{up} ; target node k ; $M = 64$; baseline $X^{(0)}$

Output: forecast $y_{\text{hat}}_{(t+H)}$ in R^4 ; attribution Attr in $R^{(L \times 5)}$; completeness gap

```

1: for tau = t-L+1 to t do
2:   S_tau <- per-node features [x_tau[i], x_tau[5]] in R^(4x2)
3:   H1 <- ReLU(S_tau W_self0 + A_down S_tau W_down0 + A_up S_tau W_up0 + b0)
   in R^(4x32)
4:   H2 <- ReLU(H1 W_self1 + A_down H1 W_down1 + A_up H1 W_up1 + b1) + E in
   R^(4x32) (E: echelon embedding)
5: end for
6: for each node i do // LSTM weights shared across nodes
7:   h(i) <- LSTM(H2_{(t-L+1,i)}, ..., H2_{(t,i)}) in R^64
8:   y_hat(i)_{(t+H)} <- x_tau[i] + g(h(i))
9: end for
10: // Post hoc attribution for target node k
11: X^(0) <- training-mean window
12: for m = 1 to M do
13:   X^(m) <- X^(0) + (m/M)(X - X^(0))
14:   G^(m) <- ∂ y_hat(k)_{(t+H)} / ∂ X^(m)
15: end for
16: Attr <- (X - X^(0)) ⊙ (1/M) SUM_m G^(m)
17: gap <- |SUM Attr - (y_hat(k)(X) - y_hat(k)(X^(0)))|
18: return y_hat_{(t+H)}, Attr, gap

```

E. Training Configuration and Hyperparameter Tuning

The model is implemented in PyTorch; the graph layers are written natively (no PyTorch Geometric). The four neural models (LSTM, ST-GCN symmetric, ST-GCN directed and the scalar paper-style hybrid) were trained with five seeds (42–46) in a Google Colab session on an NVIDIA A100 GPU (40 GB), batch size 256, with Adam [20] (learning rate $1e-3$, no weight decay), gradient-norm clipping at 1.0, at most 50 epochs and early stopping with patience 10 on validation loss. Training took 140–638 s per run. Reported metrics are means $\pm$ sample standard deviation over the five seeds; Persistence and Ridge-AR(10) are deterministic. Inference and attribution timings in Tables III and VI were measured on a CPU (PyTorch 2.12, four threads) and vary from run to run.

TABLE III
HYPERPARAMETERS AND MEASURED TIMINGS

Hyperparameter / Parameter	Configured Value	Operational Rationale
Lookback Window (L)	10 steps (20 minutes nominal)	Window length for every model
Lookahead Horizon (H)	5 rows (nominal 10 minutes)	Median 10.0 min, mean 10.8 min in the cleaned data
Graph Layers	2 layers, 32 channels	Two hops let the supplier reach the distributor
LSTM (ST-GCN models)	2 layers, hidden 64, dropout 0.2	Weights shared across the four nodes
Adam Learning Rate / Weight Decay	$1e-3$ / none	Early stopping, patience 10
IG Steps (M)	64	Completeness gap reported in Table VI
Inference Latency (CPU, batch 1)	about 2.5 ms	Measured; varies between runs

F. Continuous and Categorical Evaluation Criteria

For continuous tracking we report MSE, MAE, RMSE and R^2 on the mean of the four echelon forecasts (derived 4-node mean), plus per-node MSE and the skill relative to persistence, $1 - \text{MSE}/\text{MSE}_{\text{persistence}}$. For alert-style evaluation, each node's value is assigned to a Low, Medium or High tier using that node's training-set terciles, and accuracy and macro-F1 are computed over all node-window pairs. The test tiers are not balanced (Low 54,065, Medium 86,290, High 91,145 pairs), and for the Supplier the two thresholds differ by only 0.003 in scaled units. We also report error separately on shock windows (the top 5% of windows by mean $|y_{(t+H)} - y_{(t)}|$; 2,894 windows) and on the remaining calm windows.

IV. RESULTS ANALYSIS AND EXPERIMENTAL BENCHMARKS

A. Baseline Comparison Architectures

We compare against: (1) Naive Persistence, $y_{\text{hat}}_{(t+H)} = x_t$; (2) Ridge-AR(10), ridge regression [11] on the 50 flattened lookback features with the penalty tuned on the validation set ($\alpha = 10$); (3) a graph-free LSTM, a two-layer LSTM over the five input features with a joint four-output head and the same residual connection, trained for five seeds; and (4) a scalar paper-style GCN–LSTM hybrid re-implementing [10], which predicts only the mean risk index and is therefore not in Table IV. The LSTM and the ST-GCN differ in more than the graph

(a joint head versus a shared per-node LSTM), so the comparison is not a pure test of message passing.

B. Quantitative Benchmark Findings

Table IV reports test results on 57,875 windows at $H = 5$. The directed model has the lowest MSE ($2.73 \times 10^{-4} \pm 0.03 \times 10^{-4}$), 31.2% $\pm 0.7\%$ below Naive Persistence and 3.8% below the LSTM (2.84×10^{-4}). The gap to the LSTM (0.11×10^{-4}) exceeds the sum of the two standard deviations (0.06×10^{-4}), the criterion we use for claiming an improvement, but the LSTM has a slightly lower MAE and the architectures differ, so we do not attribute the gap to graph structure alone. The symmetric variant is worse and less stable ($3.30 \times 10^{-4} \pm 0.61 \times 10^{-4}$); two of its five seeds stopped at the persistence solution during training. Ridge-AR(10) improves on persistence by 10.8%. On three-tier severity, Naive Persistence is the most accurate (83.6% accuracy, 84.3% macro-F1); no learned model exceeds it and the directed model is lower ($81.3\% \pm 2.4\%$).

TABLE IV
TEST PERFORMANCE ON 57,875 WINDOWS ($H = 5$, MEAN OF FOUR NODES; MEAN $\pm$ STD OVER 5 SEEDS)

Architecture	MSE (10^{-4})	MAE (10^{-3})	RMSE (10^{-2})	R ² Score	Accuracy (%)	Macro-F1 (%)
Naive Persistence	3.97	5.493	1.992	0.9613	83.6	84.3
Ridge-AR(10)	3.54	6.351	1.881	0.9655	79.4	79.9
Graph-free LSTM	2.84 ± 0.03	5.118 ± 0.093	1.684 ± 0.010	$0.9723 \pm 80.5 \pm 1.6$	81.1 ± 1.6	81.1 ± 1.6
ST-GCN-LSTM (symmetric)	3.30 ± 0.61	5.385 ± 0.147	1.811 ± 0.165	$0.9678 \pm 81.9 \pm 1.8$	82.5 ± 2.0	82.5 ± 2.0
ST-GCN-LSTM (directed)	2.73 ± 0.03	5.238 ± 0.259	1.652 ± 0.008	$0.9734 \pm 81.3 \pm 2.4$	81.9 ± 2.5	81.9 ± 2.5
— SupplyGuard	0.03	0.259	0.008	0.0003		2.5

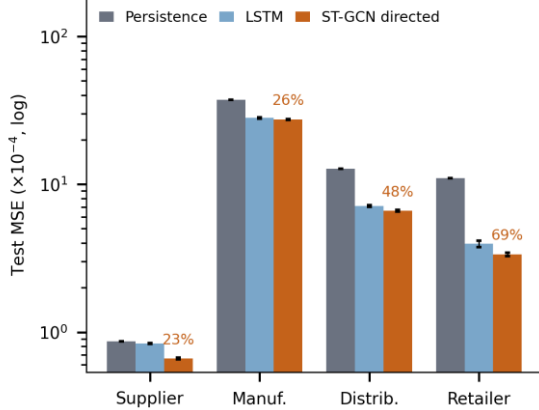


Fig. 4. Per-echelon test MSE (log scale, mean $\pm$ std over five seeds); percentages are the directed model's MSE reduction relative to persistence.

C. Continuous Trajectory Tracking Dynamics

Figure 5 shows 80 consecutive test windows around the largest Retailer shock in the test set. Persistence repeats the current value and therefore lags an abrupt change by about the forecast horizon. The directed model (five-seed mean) reacts earlier than persistence in the Retailer series, but it does not reproduce the step itself, and for the Distributor it produces a short overshoot at the onset. This is a single selected window, shown for illustration. Across the 2,894 shock windows the per-node MSE averaged over the four echelons is 0.0287 for persistence and 0.0146–0.0148 for the directed model (five seeds), about half; on the remaining calm windows persistence is better (1.2×10^{-4} versus 2.2 – 2.5×10^{-4}).

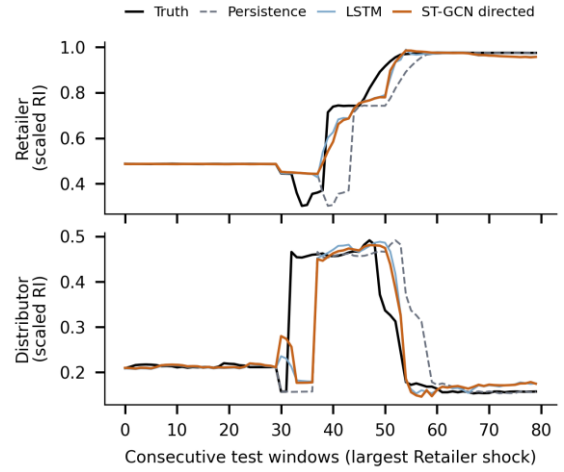


Fig. 5. Truth, Persistence, LSTM and directed ST-GCN-LSTM (five-seed means) for the Retailer and the Distributor around the largest Retailer shock in the test set.

D. Disruption Severity Classification and Alert Fidelity

Figure 6 shows the three-tier confusion matrix of the directed model (seed 42) over 231,500 node-window pairs. Row-normalised recall is 89.4% (Low), 82.6% (Medium) and 82.5% (High), and accuracy is 84.1% for this seed (five-seed mean $81.3\% \pm 2.4\%$). Errors occur mainly between adjacent tiers: 10,314 Medium cases are predicted High and 15,460 High cases are predicted Medium.

	Low	Medium	High
Actual tier			
Low	48,359 (89.4%)	5,155 (9.5%)	551 (1.0%)
Medium	4,692 (5.4%)	71,284 (82.6%)	10,314 (12.0%)
High	528 (0.6%)	15,460 (17.0%)	75,157 (82.5%)
	Low	Medium	High
	Predicted tier		

Fig. 6. Three-tier severity confusion matrix of the directed ST-GCN-LSTM (seed 42): counts and row-normalised shares.

TABLE V
PER-NODE TEST MSE AND SKILL RELATIVE TO PERSISTENCE (MEAN $\pm$ STD OVER 5 SEEDS)

Supply Chain Node	Persistence MSE (10^{-4})	LSTM MSE (10^{-4})	ST-GCN-Dir MSE (10^{-4})	Dir. skill vs persistence (%)
Tier-1 Supplier (Node S)	0.87	0.84 ± 0.01	0.66 ± 0.01	23.4
Manufacturer (Node M)	37.39	28.11 ± 0.27	27.49 ± 0.19	26.5
Regional Distributor (Node D)	12.76	7.14 ± 0.12	6.63 ± 0.09	48.0
Final Retailer (Node R)	10.98	3.96 ± 0.20	3.37 ± 0.09	69.3
Mean of the four nodes	15.50	10.01	9.54	38.5

V. OPERATIONAL DISCUSSION, EXPLAINABILITY, AND ABLATION INSIGHTS

A. How Much Does the Model Add Over Persistence?

Persistence is a strong baseline here: R^2 is 0.961 for the mean risk index and adjacent observations are correlated at $r = 0.97$ – 0.996 . The learned models nonetheless reduce the MSE by 28–31%. The reduction is not uniform: it is largest for the Retailer (69%) and the Distributor (48%) and smaller for the Supplier (23%) and the Manufacturer (26%). It comes from shock windows, where the error is about half of persistence's; on calm windows the learned correction roughly doubles the error. A practical system might therefore use persistence in calm periods and the learned forecast when a change is detected; we have not tested such a hybrid. The graph-free LSTM reaches most of the gain (28.5% versus 31.2%). The directed model's MSE is below the LSTM's by more than the sum of the two seed standard deviations at every node; the largest relative difference is at the Supplier (0.84×10^{-4} versus 0.66×10^{-4}). Because the architectures differ in more than the graph, we regard graph structure as promising rather than proven.

B. Integrated Gradients Attribution

Figure 7 summarises attribution shares for each target node, as magnitude-weighted shares of $|\text{attribution}|$ over 100 evenly spaced test windows and five seeds. In the forecast, a node's own current risk index dominates (53–74%), followed by Total Cost (18–37%); upstream echelons contribute little (the Supplier contributes 0.9% to the Distributor forecast and essentially 0% to the Retailer forecast). This largely reflects the residual connection, which copies each node's own current value into its forecast. Attributing only the learned correction shifts weight to Total Cost (37–55%); for the Supplier and Retailer targets the correction is exactly zero in 73% and 35% of windows, so those shares rest on the remaining windows. We therefore cannot support a claim that supplier variation drives downstream risk. Attributions measure the local sensitivity of the trained model, not causal effects.

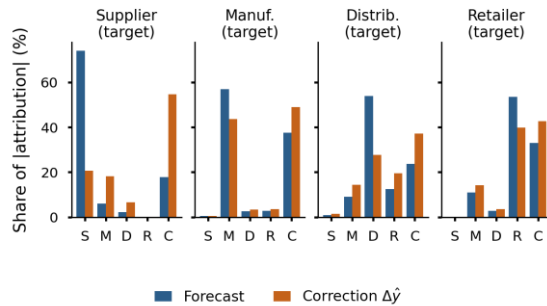


Fig. 7. Share of $|\text{Integrated Gradients attribution}|$ by input (S, M, D, R: risk indices; C: Total Cost) for each target node, for the forecast (blue) and for the learned correction (orange); 100 test windows $\times$ 5 seeds.

TABLE VI
INTEGRATED GRADIENTS FIDELITY CHECKS

Check	Observed	Reference	Status
Completeness gap, $ \text{SUM Attr} - \Delta f $ (median / 95th pct / max; $n = 2,000$)	$9.4 \times 10^{-5} / 3.1 \times 10^{-3} / 9.6 \times 10^{-2}$	Ideal: 0 ($M = 64$, Riemann sum)	Small median, heavy tail
Deletion test: top-attributed vs random input (10 high-correction windows)	7 of 10 pass	Top input should matter more than a random input	Windows selected for large corrections
Implementation invariance	Not tested	—	Not verified
Attribution latency (CPU, one target, 64 steps)	about 36 ms	—	Measured; varies between runs

C. Operational Use and Cautions

- **Early warning on shock windows:** Because the gain over persistence is concentrated on shock windows, the forecast is most useful as a change detector, not as a continuous replacement for persistence.
- **Node-level triage:** Four concurrent forecasts show which echelon is predicted to deteriorate; attributions show which inputs the model is sensitive to but do not establish cause.
- **Alert quality:** On three-tier severity, persistence is more accurate than the learned models in our evaluation (83.6% versus 81.3%), so alert thresholds should be validated before operational use.

D. Threats to Validity and Technical Limitations

We identify the following limitations. (1) Only one linear four-echelon topology ($S \rightarrow M \rightarrow D \rightarrow R$) is evaluated. (2) The data are offline records from a single dataset, with no 2017 records. (3) Gradient attributions indicate sensitivity, not causal effects. (4) The graph-free LSTM differs from the ST-GCN in more than the graph, and a control with the same architecture and an identity adjacency was not run. (5) Severity tiers are defined by per-node terciles; the Supplier's tiers are very narrow. (6) The symmetric variant was unstable across seeds. (7) Results use one chronological split, with test data from Sep–Dec 2018. (8) The horizon is five rows, which is not always exactly ten minutes. (9) The deletion test was run on ten windows selected for large corrections, not on a random sample.

VI. CONCLUSION AND FUTURE SCOPE

A. Conclusion

We presented SupplyGuard, a spatio-temporal model that forecasts the risk indices of four supply chain echelons concurrently. On the Mendeley SCRM dataset, the directed model reduced the mean-node MSE to 2.73×10^{-4} ($R^2 = 0.973$), 31% below Naive Persistence and 4% below a graph-free LSTM, with the benefit concentrated on shock windows. Persistence, however, achieved higher three-tier severity accuracy, and the ablation is confounded, so we do not claim that graph structure alone explains the gain. Integrated Gradients attributions have a median completeness gap of 9.4×10^{-5} with a heavy tail and describe the model's local sensitivity rather than causal effects; in our results each node's forecast is dominated by its own current value and the Total Cost, not by upstream echelons.

B. Future Work

- **Dynamic Mesh Topologies:** Extend to multi-supplier networks and test attention-based edge weights.

- **External Telemetry:** Add external signals such as weather or port congestion where data are available.
- **Controlled Ablation and Hybrid Alerts:** Run an identity-adjacency control with the same architecture, and test a hybrid that uses persistence in calm periods and the learned correction on detected shocks.

ACKNOWLEDGMENT

The authors thank their project supervisor, Mrs. N. Chandana, Assistant Professor, Department of Computer Science and Engineering, Anurag University, Hyderabad, for her guidance throughout this work, and the authors of the Mendeley time-series dataset [3] for making it publicly available.

REFERENCES

- [1] D. Ivanov, A. Dolgui, and B. Sokolov, "The impact of digital technology and Industry 4.0 on the ripple effect and supply chain risk analytics," *Int. J. Prod. Res.*, 2018, doi: 10.1080/00207543.2018.1488086.
- [2] D. Simchi-Levi, W. Schmidt, and Y. Wei, "From superstorms to factory fires: Managing unpredictable supply-chain disruptions," *Harvard Business Review*, vol. 92, no. 1–2, pp. 96–101, 2014.
- [3] H. Banerjee, G. Saparia, V. Ganapathy, P. Garg, and V. M. Shenbagaraman, "Time series dataset for risk assessment in supply chain networks," *Mendeley Data*, V2, 2019, doi: 10.17632/gystn6d3r4.2.
- [4] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997. doi: 10.1162/neco.1997.9.8.1735.
- [5] B. Yu, H. Yin, and Z. Zhu, "Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting," in *Proc. 27th Int. Joint Conf. Artif. Intell. (IJCAI)*, 2018, pp. 3634–3640. doi: 10.24963/ijcai.2018/505.
- [6] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. 5th Int. Conf. Learn. Represent. (ICLR)*, 2017.
- [7] A. Dolgui, D. Ivanov, and B. Sokolov, "Ripple effect in the supply chain: An analysis and recent literature," *Int. J. Prod. Res.*, vol. 56, no. 1–2, pp. 414–430, 2018, doi: 10.1080/00207543.2017.1387680.
- [8] M. Sundararajan, A. Taly, and Q. Yan, "Axiomatic attribution for deep networks," in *Proc. 34th Int. Conf. Mach. Learn. (ICML)*, 2017, pp. 3319–3328.
- [9] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu, "A comprehensive survey on graph neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 1, pp. 4–24, Jan. 2021. doi: 10.1109/TNNLS.2020.2978386.
- [10] I. Farzhana and L. Dev Harris, "Hybrid GNN-LSTM model for real-time supply chain risk prediction," in *Proc. 8th Int. Conf. Comput. Methodologies Commun. (ICCMC)*, IEEE, 2025, doi: 10.1109/ICCMC65190.2025.11140739.
- [11] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
- [12] H. Min, "Artificial intelligence in supply chain management: Theory and applications," *Int. J. Logist. Res. Appl.*, vol. 13, no. 1, pp. 13–39, 2010.
- [13] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph attention networks," in *Proc. 6th Int. Conf. Learn. Represent. (ICLR)*, 2018.
- [14] R. Caruana, "Multitask learning," *Mach. Learn.*, vol. 28, no. 1, pp. 41–75, 1997. doi: 10.1023/A:1007379606734.
- [15] J. Adebayo, J. Gilmer, M. Muelly, I. Goodfellow, M. Hardt, and B. Kim, "Sanity checks for saliency maps," in *Proc. 32nd Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2018, pp. 9505–9515.
- [16] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, "The graph neural network model," *IEEE Trans. Neural Netw.*, vol. 20, no. 1, pp. 61–80, Jan. 2009.
- [17] Y. Li, R. Yu, C. Shahabi, and Y. Liu, "Diffusion convolutional recurrent neural network: Data-driven traffic forecasting," in *Proc. 6th Int. Conf. Learn. Represent. (ICLR)*, 2018.
- [18] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 770–778.
- [19] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Proc. 31st Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2017, pp. 4765–4774.
- [20] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. 3rd Int. Conf. Learn. Represent. (ICLR)*, 2015.